Practical 10: Recursion and Backtracking

Aim

To study and implement recursion using a Factorial program and backtracking using the N-Queens problem in Python.

Theory

Recursion is a programming technique where a function calls itself, directly or indirectly, to solve smaller instances of the same problem, until it reaches a base case that can be solved directly without further recursion; the factorial function demonstrates this, with factorial(0) = 1 as the base case and factorial(n) = n * factorial(n-1) as the recursive case, with each call pushing a new frame onto the call stack until the base case unwinds the recursion. Backtracking is an algorithmic technique for solving constraint-satisfaction problems incrementally, by building a solution one step at a time and abandoning ('backtracking' from) a partial solution as soon as it is determined that it cannot possibly lead to a valid complete solution, which prunes the search space far more efficiently than checking every possible combination (brute force). The N-Queens problem is a classic backtracking example: it requires placing N queens on an N x N chessboard such that no two queens attack each other (i.e., no two share the same row, column, or diagonal); the algorithm places a queen in a row, checks safety against all previously placed queens using is_safe(), recurses to the next row if safe, and backtracks (removes the queen and tries the next column) if no column in the current row is safe.

Algorithm

1. Factorial: if n is 0 return 1, otherwise return n multiplied by factorial(n-1).

2. N-Queens: maintain a board array storing the column position of the queen in each row.

3. Define is_safe() to check if placing a queen at a given row/column conflicts with previously placed queens.

4. Define backtrack() which tries placing a queen in each column of the current row; if safe, recurse to the next row, otherwise backtrack.

5. When all rows are filled, save the solution.

6. Print the factorial result and the number/first solution of the N-Queens problem.

Program

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

def solve_nqueens(n):
    solutions = []
    board = [-1] * n

    def is_safe(row, col):
        for r in range(row):
            if board[r] == col or abs(board[r] - col) == abs(r - row):
                return False
        return True

    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1
    backtrack(0)
```

```
    return solutions

print("Factorial of 5:", factorial(5))

solutions = solve_nqueens(4)
print(f"Number of solutions for 4-Queens: {len(solutions)}")
print("First solution (col positions per row):", solutions[0])
```

Output

```
Factorial of 5: 120
Number of solutions for 4-Queens: 2
First solution (col positions per row): [1, 3, 0, 2]
```

Result / Conclusion

Recursion and backtracking were successfully implemented and demonstrated using the Factorial and N-Queens programs.

Viva Questions

Q1. What are the two essential components of every recursive function?

Ans: 1. Base case: a condition under which the function returns a result directly without calling itself (e.g., factorial(0) = 1). It stops the recursion and prevents infinite loops. 2. Recursive case: the function calls itself with a simpler/smaller version of the problem (e.g., n * factorial(n-1)), making progress toward the base case.

Q2. What is a stack overflow in the context of recursion?

Ans: Each recursive call adds a new frame (with local variables and the return address) to the call stack. If recursion is too deep (no base case, or n is very large), the call stack exhausts available memory, causing a stack overflow (RecursionError in Python: 'maximum recursion depth exceeded'). Python's default recursion limit is 1000. sys.setrecursionlimit() can increase it, but deep recursion should be converted to iteration for safety.

Q3. What is backtracking and how does it differ from brute force?

Ans: Brute force generates all possible solutions and checks each one — extremely slow for large search spaces. Backtracking builds the solution incrementally and abandons (backtracks from) a partial solution as soon as a constraint violation is detected, without exploring the remaining subtree. This pruning makes backtracking much faster than brute force for constraint-satisfaction problems like N-Queens, Sudoku, and crossword puzzles.

Q4. How does the is_safe() function check for diagonal conflicts in N-Queens?

Ans: Two queens at (r1, c1) and (r2, c2) are on the same diagonal if and only if |r1 - r2| == |c1 - c2| (the row distance equals the column distance). is_safe(row, col) checks for three conflicts with each previously placed queen at board[r]: same column (board[r] == col), or same diagonal (abs(board[r] - col) == abs(r - row)). Row conflicts are impossible since each row places exactly one queen.

Q5. What is the difference between direct and indirect recursion?

Ans: Direct recursion: a function calls itself directly (e.g., factorial calls factorial). Indirect recursion: function A calls function B which calls function A (e.g., isEven(n) calls isOdd(n-1) which calls isEven(n-2)). Both require proper base cases to terminate. Mutual recursion (indirect) is sometimes used for alternating state machines but is generally harder to reason about and optimise.